

Guia Final de Execução

Injeção de Contexto de Tarefa no Preditor de Intenção

Marcos Kalile

Projeto PIBIC/CNPq — IME/LIARC

Orientador: Prof. Hebert Azevedo-Sá

8 de julho de 2026

Leia isto primeiro

Todo o código (OS-1 a OS-7) já está pronto e testado (16/16 testes automatizados). O que falta é executar 4 blocos, nesta ordem: **(1) gravar e anotar sessões, (2) consolidar 3 datasets, (3) treinar V0/V1/V2, (4) gerar tabela e figuras finais**. Este guia é um roteiro de execução, não uma justificativa de decisões de projeto (essas já estão no ROADMAP_IC.pdf). Cada seção abaixo diz exatamente o que fazer e qual comando rodar.

Sumário

1	Visão geral: os 4 blocos	2
2	Bloco 1 — Gravar e anotar as sessões	2
2.1	Bancada (montar uma vez, não mexer mais)	2
2.2	As 6 rotas canônicas	2
2.3	Quantas sessões gravar	3
2.4	Como gravar cada sessão	3
2.5	Como anotar cada sessão	4
3	Bloco 2 — Consolidar os 3 datasets	4
3.1	Checklist de sanidade (2 minutos, não pule)	5
4	Bloco 3 — Treinar V0, V1, V2	5
4.1	Comandos, na ordem	5
5	Bloco 4 — Tabela final e matrizes de confusão	6
5.1	Exemplo de figura (dados sintéticos, só para conferir o formato)	6
5.2	Como ler o resultado	6
6	Checklist único	7
A	Rotas detalhadas, passo a passo	7
A.1	R01 — Tarefa completa (rota canônica)	8
A.2	R02 — Estágio <code>bottom</code> isolado (rota mínima)	8
A.3	R03 — <code>bottom</code> → <code>four_tubes</code> (sem <code>top/wheels</code>)	8
A.4	R04 — Retomada via <code>-stageI_done</code>	8
A.5	R05 — Completa com <code>no_action</code> intercalado	9
A.6	R06 — Ordem alternativa	9

1 Visão geral: os 4 blocos

Bloco	O que fazer	Ferramenta	Tempo estimado	
1	Gravar e anotar 12 sessões (Seção 2)	<code>capture_session</code> , <code>annotate_pk1</code>	1–2 dias	
2	Consolidar 3 datasets de contexto (Seção 3)	<code>build_json</code>	15 min	
3	Treinar V0, V1, V2 (Seção 4)	<code>train_finetune.py</code>	algumas horas (CPU)	
4	Gerar tabela e matrizes de confusão (Seção 5)	<code>aggregate_results.py</code>	5 min	

Ambiente usado em todos os comandos: `conda hrc (source /home/marcos-kalile/anaconda3/bin/activate hrc)`. Coleta e consolidação rodam em `hrc-data-collection/`; treino e agregação rodam em `hrc-finetune/`.

2 Bloco 1 — Gravar e anotar as sessões

2.1 Bancada (montar uma vez, não mexer mais)

- 8 conectores, 12 parafusos, 4 rodas, 8 tubos curtos (~ 12 cm), 4 tubos longos (~ 25 cm);
- conectores à direita; parafusos à esquerda perto; rodas à esquerda atrás dos parafusos; gabarito ao centro; marca de repouso na borda;
- 40–50 cm entre zonas.

Não mude entre sessões

Câmera, posição das zonas, cadeira, iluminação: fixas do início ao fim das 12 sessões. Só variam ordem de pegas, ritmo e postura inicial.

2.2 As 6 rotas canônicas

As rotas abaixo vêm do documento `Rotas_e_Vetor_de_Contexto.pdf` (política `proxy_graph`, a mesma que `plan_sim.py` já usa em produção) e foram validadas contra o simulador — os passos e o contexto 7D exibidos por rota já são a saída real de `plan_sim.py`, não uma estimativa. Use-as como roteiro de gravação.

Rota	Papel	Sequência	Passos
R01	Tarefa completa (canônica)	<code>bottom</code> → <code>four_tubes</code> → <code>top</code> → <code>wheels</code> . Fecha com contexto <code>[1000 1.00, 1.00, 1.00]</code> . Esta é a rota de referência.	29
R02	Estágio <code>bottom</code> isolado	Só a primeira parte de R01 (4 conectores + 4 parafusos de <code>bottom</code>), sem prosseguir. Rota mínima de validação.	8
R03	<code>bottom</code> → <code>four_tubes</code>	Prefixo de R01 até o fim de <code>four_tubes</code> , sem <code>top/wheels</code> .	17
R04	Retomada via <code>-stageI_done</code>	Começa direto em <code>four_tubes</code> (<code>bottom</code> já assumido pronto), segue <code>top</code> → <code>wheels</code> . Usa o preset de retomada do simulador.	21

Rota	Papel	Sequência	Passos
R05	Completa com <code>no_action</code> intercalado	Mesma sequência de R01, mas com pausas de <code>no_action</code> inseridas em pontos de contexto igual (engorda a classe <code>no_action</code> sem inventar contexto novo).	32
R06	Ordem alternativa	<code>bottom</code> → <code>top</code> → <code>four_tubes</code> → <code>wheels</code> (troca a posição de <code>top</code> e <code>four_tubes</code> em relação a R01). Também válida no grafo.	29

Como usar as 6 rotas para montar as sessões

- **R01 primeiro, sempre:** grave e anote só ela, rode o Bloco 2 com `-allow-empty-split` para validar a cadeia inteira antes de investir nas demais.
- **Diversidade estrutural** (muda a sequência de estágios, robustece o vetor de contexto contra viés de ordem): alterne R01 e R06 nas repetições de treino.
- **Diversidade de trajetória** (mesmo grafo, mesmo contexto, esqueleto diferente): use R05 para gerar exemplos de `no_action` com o mesmo par (contexto, próxima intenção) que R01, variando só a execução física (pausas, micro-variações de postura, ritmo).
- **R02/R03/R04 são rotas curtas**, úteis para engordar classes específicas (`bottom` isolado, transição para `four_tubes`, retomada) sem repetir a tarefa inteira — gravação mais rápida quando faltar janela de uma classe pontual.
- Recomendação do documento de rotas: execute R01/R06 com variação natural (ritmo, postura) e **repita cada uma 8–12 vezes** para densificar amostras por par (intenção, estágio). Isso substitui a ideia de "12 sessões distintas com rótulos artificiais" por "poucas rotas bem definidas, repetidas com variação natural".

2.3 Quantas sessões gravar

Sessões	O que gravar	Split
1. ^a	R01 (piloto — valida a cadeia antes de continuar)	treino
2. ^a –5. ^a	Alterne R01/R06 (2 de cada), ritmo e postura naturais entre repetições	treino
6. ^a –8. ^a	R05 (completa com <code>no_action</code> intercalado), variando ritmo	treino
9. ^a	R02, R03 ou R04 — só a que estiver mais fraca após o Bloco 2	treino
10. ^a	R01 completa, ritmo normal. Sessão inteira reservada para teste — não editar depois de gravar.	teste
11. ^a	R06 completa, ritmo normal. Segunda sessão de teste.	teste
12. ^a (buffer)	Só grave se o relatório do Bloco 2 (Seção 3.1) mostrar alguma classe com <120 janelas. Repita a rota que cobre essa classe.	a definir

2.4 Como gravar cada sessão

Repita este bloco de comandos para cada uma das 11–12 sessões da tabela acima. Troque `-script-id` pela rota da linha atual (R01, R06, R05, etc.).

```
cd /home/marcos-kalile/hrc-data-collection
```

```
source .venv/bin/activate

# ID sugerido automaticamente (formato SNN_YYYYMMDD)
python -m datacol.capture_session --suggest-session-id

# Gravar (troque --script-id pela rota desta sessão: R01, R06, R05, ...)
python -m datacol.capture_session \
  --participant P01 --script-id R01 \
  --camera-model "Intelbras WCI 1080p" --camera-distance-m 2.2 \
  --width 1920 --height 1080 --fps 30

# Validar logo em seguida
python -m datacol.capture_session --validate-session sessions/<ID_GERADO>
```

Regra de execução do gesto, sempre: parado no repouso → mão até a zona → pega e deposita → volta ao repouso → pausa breve → próximo gesto. Nunca encadeie dois gestos sem passar pelo repouso. Varie o ritmo e a postura de repouso naturalmente entre repetições da mesma rota (isso já cobre a diversidade de trajetória sem precisar de rótulos extras de "rápido"/"lento").

2.5 Como anotar cada sessão

Anote logo depois de gravar (não acumule). Para cada sessão:

```
python -m datacol.annotate_pkl sessions/<ID_GERADO>
```

Por gesto: Espaço pausa/toca → **b** no primeiro quadro do gesto → navegue até o último quadro → tecla da classe (1=conectores, 2=parafusos, 3=rodas, **i**=ignore). No bloco dos 4 tubos curtos de **four_tubes**: marque tudo como **ignore**, volte ao primeiro quadro do bloco e pressione **f**. Pressione **s** para salvar — o programa recusa se houver lacuna ou sobreposição.

Contagem esperada por sessão R01/R06 (tarefa completa): 8 **get_connectors**, 12 **get_screws**, 4 **get_wheels**, 9 **ignore**, 1 evento **begin_four_tubes**. Rotas curtas (R02, R03, R04) têm contagens proporcionalmente menores — confira contra a coluna "Passos" da Seção 2.2 (cada passo da tabela é uma ação/intenção; **no_action** e **ignore** não entram nessa contagem).

Toda sessão com four_tubes (R01, R03, R04, R05, R06): antes de consolidar, rode `python -m datacol.context_replay sessions/<ID> -context-dim 7` e confirme visualmente que o estágio muda no quadro certo.

3 Bloco 2 — Consolidar os 3 datasets

Depois de anotar todas as sessões, gere os três JSONs (`context_dim` 0, 7, 10) **numa única rodada de comandos**, sempre com os mesmos IDs de teste:

```
cd /home/marcos-kalile/hrc-data-collection
source .venv/bin/activate

for dim in 0 7 10; do
  python -m datacol.build_json \
    --sessions-root sessions \
    --output datasets/v1/dataset_dim${dim}.json \
    --report datasets/v1/report_classes_dim${dim}.md \
    --test-session <ID_SESSAO_10_R01> \
    --test-session <ID_SESSAO_11_R06> \
    --context-dim ${dim} \
    --window-size 5
done
```

Troque `<ID_SESSAO_10_R01>/<ID_SESSAO_11_R06>` pelos IDs reais das duas sessões de teste (10.^a e 11.^a da Seção 2.3, formato SNN_YYYYMMDD).

3.1 Checklist de sanidade (2 minutos, não pule)

Abra os três `report_classes_dim*.md` e confira:

Confirme antes de treinar

1. Contagem de janelas por classe é **igual** nos três relatórios.
2. Nenhuma classe com zero janelas em nenhum split.
3. Cada classe com ≥ 120 janelas (treino+teste). Se faltar $\rightarrow$ grave R12 focado nessa classe e repita o Bloco 2.
4. `no_action` não deve passar de $\sim 50\%$ do total.

4 Bloco 3 — Treinar V0, V1, V2

Variante	O que é	Para que serve
V0	Checkpoint original, sem treinar	mede o gap webcam vs. dados originais
V1	Fine-tune sem contexto	mede o ganho só do fine-tuning
V2	Fine-tune com contexto, partindo de V1	resultado principal: delta V2–V1

Regra fixa

Treine nesta ordem: V0 $\rightarrow$ V1 $\rightarrow$ V2. V2 sempre parte do checkpoint de V1 (nunca de V0), e cada seed de V2 parte do **mesmo número de seed** em V1 (seed 0 de V2 vem do seed 0 de V1, etc.).

4.1 Comandos, na ordem

```
cd /home/marcos-kalile/hrc-finetune
source /home/marcos-kalile/anaconda3/bin/activate hrc

CKPT=/home/marcos-kalile/IC_Kalile_Intention_Prediction_HRC/traj_intention/checkpoints/
seq5_pred5_epoch40_whole_pkl_final_intention_nomaskTrue.pth
DS=/home/marcos-kalile/hrc-data-collection/datasets/v1

# --- V0: so avalia o checkpoint original ---
python train_finetune.py --variant V0 \
  --dataset $DS/dataset_dim0.json --context-dim 0 \
  --init-checkpoint $CKPT --out-dir runs/V0

# --- V1: fine-tune sem contexto, 3 sementes ---
python train_finetune.py --variant V1 \
  --dataset $DS/dataset_dim0.json --context-dim 0 \
  --init-checkpoint $CKPT --seeds 0 1 2 \
  --epochs 10 --lr 1e-4 --batch-size 32 --out-dir runs/V1

# --- V2: fine-tune com contexto 7D, partindo de V1 (mesma seed) ---
python train_finetune.py --variant V2 \
  --dataset $DS/dataset_dim7.json --context-dim 7 \
  --init-checkpoint-per-seed runs/V1/seed0/checkpoint.pth \
  runs/V1/seed1/checkpoint.pth \
  runs/V1/seed2/checkpoint.pth \
```

```

--seeds 0 1 2 --epochs 10 --freeze-epochs 4 \
--lr 1e-4 --context-lr 1e-3 --batch-size 32 --out-dir runs/V2_dim7

# --- V2 (10D), so se sobrar tempo: ablacao opcional ---
python train_finetune.py --variant V2 \
--dataset $DS/dataset_dim10.json --context-dim 10 \
--init-checkpoint-per-seed runs/V1/seed0/checkpoint.pth \
                                runs/V1/seed1/checkpoint.pth \
                                runs/V1/seed2/checkpoint.pth \
--seeds 0 1 2 --epochs 10 --freeze-epochs 4 \
--lr 1e-4 --context-lr 1e-3 --batch-size 32 --out-dir runs/V2_dim10

```

Cada rodada grava checkpoint + `metrics.json` por seed, mais um `summary.json` com média $\pm$ desvio-padrão. Se o dataset final tiver poucas janelas por classe, baixe `-batch-size` para 8–16.

Por que não há perda de trajetória

O treino original usa duas perdas (classificação + regressão de trajetória). O dataset novo (`build_json.py`) só grava a janela de entrada e o rótulo — não o alvo de trajetória futura — então `train_finetune.py` treina só com a perda de classificação. Isso é proposital: o objetivo é isolar o efeito do contexto na classificação, não reproduzir a tarefa de trajetória do artigo original.

5 Bloco 4 — Tabela final e matrizes de confusão

```

cd /home/marcos-kalile/hrc-finetune
source /home/marcos-kalile/anaconda3/bin/activate hrc
pip install scikit-learn matplotlib -q

python reports/scripts/aggregate_results.py \
--runs-root runs --out-dir results

```

Gera `results/ablation_table.md` (tabela V0/V1/V2 com acurácia, ECE e o delta V2–V1) e `results/confusion_matrices.png` (uma matriz por variante, lado a lado).

5.1 Exemplo de figura (dados sintéticos, só para conferir o formato)

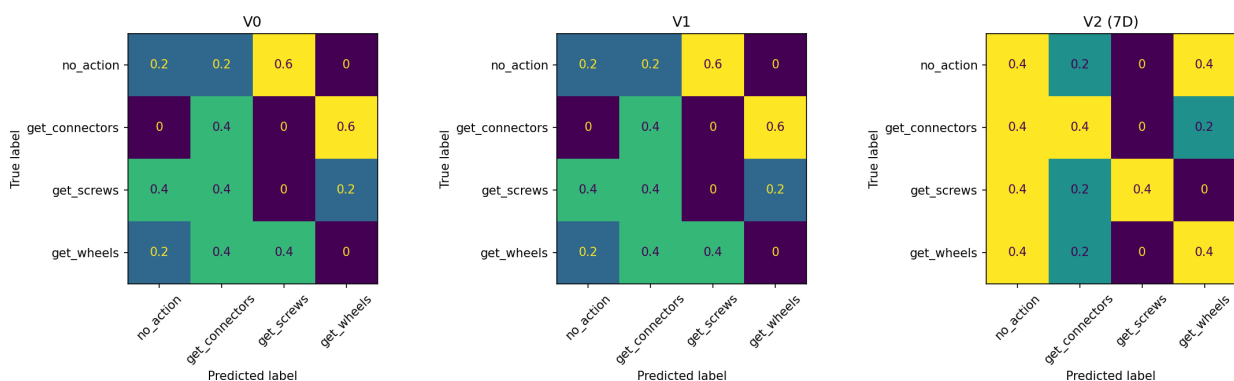


Figura 1: Formato esperado da figura final — substitua rodando o script acima com `runs/` real.

5.2 Como ler o resultado

- Diagonal da matriz = acurácia por classe (cada linha soma 1).

- $V0 \rightarrow V1$: melhora mede a adaptação de domínio (fine-tuning puro).
- $V1 \rightarrow V2$: melhora adicional é o efeito do contexto — **este é o resultado central do artigo**.
- Metas de referência do roadmap: $\text{top-1} \geq 70\%$ e $\text{ECE} < 0,08$ em V2. Se não bater, o resultado ainda é publicável: o que importa é o delta $V2-V1$, não o valor absoluto.

6 Checklist único

Do início ao fim

1. Gravar + anotar a 1.^a sessão (rota R01). Rodar Bloco 2 com `-allow-empty-split` para validar a cadeia.
2. Gravar + anotar as demais 9–10 sessões de treino (rotas R01/R06/R05/R02/R03/R04 conforme a Seção 2.3), anotando cada uma logo após gravar.
3. Gravar + anotar as 2 sessões de teste (rotas R01 e R06, ritmo normal, sessão inteira reservada).
4. Rodar `context_replay` em toda sessão com `four_tubes` (R01, R03, R04, R05, R06).
5. Rodar Bloco 2 completo (3 datasets) com os IDs reais das 2 sessões de teste.
6. Checar os 3 relatórios de classes (Seção 3.1). Se faltar janela, gravar a 12.^a sessão buffer e repetir.
7. Rodar V0, depois V1 (3 sementes), depois V2 7D (3 sementes pareadas). V2 10D se sobrar tempo.
8. Rodar `aggregate_results.py`.
9. Escrever a seção de resultados do artigo com `ablation_table.md` e `confusion_matrices.png`.

Onde está cada arquivo

- Coleta/anotação/consolidação: `hrc-data-collection/src/datacol/`
- Treino e inferência: `hrc-finetune/{train_finetune,DLinear,predict}.py`
- Testes: `hrc-finetune/tests/`
- Agregação de resultados: `hrc-finetune/reports/scripts/aggregate_results.py`
- Este guia: `hrc-finetune/reports/relatorio_final_ic.tex`

A Rotas detalhadas, passo a passo

Este apêndice traz o roteiro completo de cada rota, extraído de `Rotas_e_Vetor_de_Contexto.pdf` (saída real de `plan_sim.py`, política `proxy_graph`). Use como guia literal de gravação: cada linha é um gesto a executar na bancada, na ordem exata. **Contexto 7D** = one-hot de estágio `[none,bottom,four_tubes,top]` || conectores/8, parafusos/12, rodas/4.

Convenções das tabelas: **Gesto** é o que fazer com as mãos (classe a rotular depois, ou *ignore*/transição); **Estágio** é o estágio ativo após o gesto; **Parafusos** mostra a contagem acumulada

por estágio no formato **b**=bottom, **ft**=four_tubes, **t**=top (ex. **b2 ft0 t0** = 2 parafusos em bottom, 0 nos demais).

A.1 R01 — Tarefa completa (rota canônica)

Ordem: bottom → four_tubes → top → wheels. **29 passos.** **Contexto final:** [1000 | 1.00,1.00,1.00] (tudo completo, sem estágio ativo).

#	Gesto	Estágio	Parafusos	Contexto 7D após o gesto
1–4	get_connectors ×4 (2 curtos + 2 longos) em bottom	bottom → none	b0 ft0 t0	sobe de [0,1,0,0 0.12,0,0] até [1,0,0,0 0.50,0,0]
5–8	get_screws ×4 em bottom	none	b1→b4 ft0 t0	parafusos sobe 0.08 por vez, de 0.42 até [1000 0.50,0.33,0]
9	<i>begin_four_tubes</i> (transição, marcar f na anotação)	four_tubes	b4 ft0 t0	[0,0,1,0 0.50,0.33,0]
10–13	short ×4 (colocação manual dos 4 tubos curtos, <i>ignore</i> na anotação)	four_tubes → none	b4 ft0 t0	sobe conectores até [1000 1.00,0.33,0]
14–17	get_screws ×4 em four_tubes	none	b4 ft1→ft4 t0	parafusos sobe até [1000 1.00,0.67,0]
18–21	get_connectors ×4 (2 curtos + 2 longos) em top	top → none	b4 ft4 t0	[0,0,0,1 1.00,0.67,0] durante, volta a [1000 ...] ao fechar
22–25	get_screws ×4 em top	none	b4 ft4 t1→t4	parafusos sobe até [1000 1.00,1.00,0]
26–29	get_wheels ×4 (rodas)	none	b4 ft4 t4	rodas sobe 0.25 por vez até [1000 1.00,1.00,1.00]

Ao anotar: 8 intervalos **get_connectors**, 12 **get_screws**, 4 **get_wheels**; o bloco dos 4 **short** (passos 10–13) é um único intervalo **ignore** com **f** marcado no passo 9.

A.2 R02 — Estágio bottom isolado (rota mínima)

Ordem: só bottom, sem prosseguir. **8 passos.** **Contexto final:** [1000 | 0.50,0.33,0.00].

Idêntica aos passos 1–8 de R01 (Seção A.1): 4 **get_connectors** + 4 **get_screws** em bottom, e pare. Use esta rota só para engordar rapidamente as classes **get_connectors**/**get_screws** do estágio **bottom** quando o relatório de classes mostrar déficit ali.

A.3 R03 — bottom → four_tubes (sem top/wheels)

Ordem: bottom → four_tubes, para antes do top. **17 passos.** **Contexto final:** [1000 | 1.00,0.67,0.00].

Idêntica aos passos 1–17 de R01 (Seção A.1): tudo de bottom (passos 1–8), a transição *begin_four_tubes* + 4 **short** (passos 9–13), e os 4 **get_screws** de **four_tubes** (passos 14–17). Pare aí — não faça **top** nem **wheels**. Útil para engordar a transição de estágio **bottom**→**four_tubes** isoladamente.

A.4 R04 — Retomada via -stageI_done

Ordem: começa direto em **four_tubes** (assume bottom já completo), segue **top** → **wheels**. **21 passos.** **Contexto final:** [1000 | 1.00,1.00,1.00].

#	Gesto	Estágio	Parafusos	Contexto 7D após o gesto
1	<i>begin_four_tubes</i> (transição)	four_tubes	b4 ft0 t0	[0,0,1,0 0.50,0.33,0]
2–5	<i>short</i> ×4 (<i>ignore</i>)	four_tubes → none	b4 ft0 t0	sobe até [1000 1.00,0.33,0]
6–9	<i>get_screws</i> ×4 em <i>four_tubes</i>	none	b4 ft1→ft4 t0	sobe até [1000 1.00,0.67,0]
10–13	<i>get_connectors</i> ×4 em <i>top</i>	top → none	b4 ft4 t0	volta a [1000 1.00,0.67,0] ao fechar
14–17	<i>get_screws</i> ×4 em <i>top</i>	none	b4 ft4 t1→t4	sobe até [1000 1.00,1.00,0]
18–21	<i>get_wheels</i> ×4	none	b4 ft4 t4	sobe até [1000 1.00,1.00,1.00]

Nota: esta rota pressupõe que *bottom* já está montado fisicamente na bancada (4 conectores + 4 parafusos já colocados) antes de começar a gravar — prepare a bancada nesse estado inicial ou grave R02 imediatamente antes e continue a mesma sessão a partir daí.

A.5 R05 — Completa com *no_action* intercalado

Ordem: igual a R01, com pausas de *no_action* inseridas em 3 pontos (mesmo contexto, intenção *no_action*). **32 passos.** **Contexto final:** [1000 | 1.00,1.00,1.00].

Siga exatamente R01 (Seção A.1), mas insira uma pausa parada no repouso (2 gestos *no_action* de ~2s cada) nestes três pontos:

- após o passo 4 de R01 (logo depois de fechar os 4 conectores de *bottom*, contexto [1000 | 0.50,0,0]);
- após o passo 8 de R01 (logo depois dos 4 parafusos de *bottom*, contexto [1000 | 0.50,0.33,0]);
- logo antes do passo 9 de R01 (imediatamente antes da transição *begin_four_tubes*, mesmo contexto do ponto anterior).

Essas pausas geram janelas de *no_action* com o mesmo vetor de contexto de janelas de ação real próximas — é o par "mesmo-contexto/intenção-diferente" que ajuda o modelo a não confundir contexto com rótulo.

A.6 R06 — Ordem alternativa

Ordem: *bottom* → *top* → *four_tubes* → *wheels* (troca a posição de *top* e *four_tubes* em relação a R01). **29 passos.** **Contexto final:** [1000 | 1.00,1.00,1.00].

#	Gesto	Estágio	Parafusos	Contexto 7D após o gesto
1–4	<i>get_connectors</i> ×4 em <i>bottom</i>	<i>bottom</i> → none	b0 ft0 t0	sobe até [1000 0.50,0,0]
5–8	<i>get_screws</i> ×4 em <i>bottom</i>	none	b1→b4 ft0 t0	sobe até [1000 0.50,0.33,0]
9–12	<i>get_connectors</i> ×4 em <i>top</i>	<i>top</i> → none	b4 ft0 t0	sobe até [1000 1.00,0.33,0]
13–16	<i>get_screws</i> ×4 em <i>top</i>	none	b4 ft0 t1→t4	sobe até [1000 1.00,0.67,0]
17	<i>begin_four_tubes</i> (transição)	four_tubes	b4 ft0 t4	[0,0,1,0 1.00,0.67,0]

#	Gesto	Estágio	Parafusos	Contexto 7D após o gesto
18– 21	short ×4 (<i>ignore</i>)	four_tubes → none	b4 ft0 t4	volta a [1000 1.00,0.67,0]
22– 25	get_screws ×4 em four_tubes	none	b4 ft1→ft4 t4	sobe até [1000 1.00,1.00,0]
26– 29	get_wheels ×4	none	b4 ft4 t4	sobe até [1000 1.00,1.00,1.00]

Diferença chave em relação a R01: aqui **top** é montado *antes* de **four_tubes** — observe que o one-hot de estágio passa por **top** (passos 9–16) antes de **four_tubes** (passos 17–21), o oposto da ordem de R01. É essa troca que dá diversidade estrutural ao contexto 7D.

Lembrete

Em toda rota com **four_tubes** (R01, R03, R04, R05, R06), marque a tecla **f** no `annotate.pkl.py` exatamente no primeiro quadro do bloco **ignore** dos tubos curtos — é isso que grava o evento **begin_four_tubes** em `plan_events.json` e faz o contexto calculado por `build_json.py` bater com as tabelas acima.